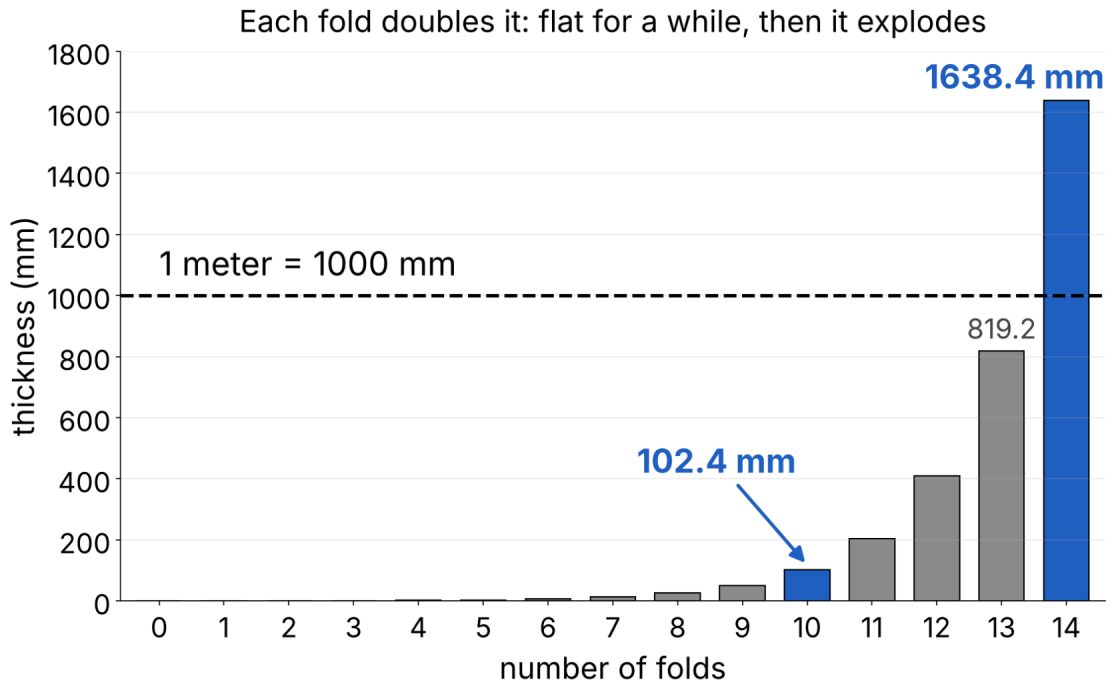
MATH FOR AI · EPISODE 0.6

Exponents and Logarithms

Repeated multiplication, and the question asked backwards.

Dr. Joe

Last time's challenge



10 FOLDS

$$0.1 \times 2^{10} = 0.1 \times 1024 = 102.4 \text{ mm}$$

About **10 cm**. Ten doublings multiply by 1024.

BONUS: PAST 1 METER?

13 folds: **819.2 mm**. Not yet.
14 folds: **1638.4 mm**. Past it!

An exponent is repeated multiplication

$$b^n = b \cdot b \cdot \cdots \cdot b \qquad (n \text{ copies of } b)$$

$$2^{10} = 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 1024$$

THE NAMES

b is the **base**. **n** is the **exponent** (or **power**).

SAY IT OUT LOUD

“2 to the 10th.” And x^2 is “x squared,” from Episode 0.4.

Three rules, all just counting

$$b^m \cdot b^n = b^{m+n} \qquad 2^3 \cdot 2^4 = (2 \cdot 2 \cdot 2)(2 \cdot 2 \cdot 2 \cdot 2) = 2^7$$

Multiplying: add the exponents. 3 twos and 4 more twos make 7 twos.

$$\frac{b^m}{b^n} = b^{m-n} \qquad \frac{2^5}{2^2} = \frac{2 \cdot 2 \cdot 2 \cdot 2 \cdot 2}{2 \cdot 2} = 2^3$$

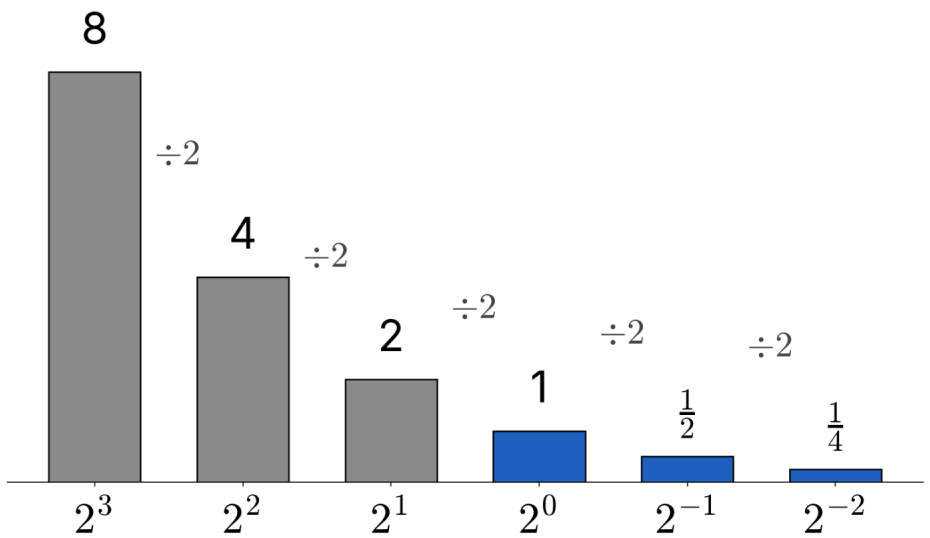
Dividing: subtract the exponents. Cancel 2 twos from top and bottom: 3 are left.

$$(b^m)^n = b^{mn} \qquad (2^3)^2 = 2^3 \cdot 2^3 = 2^6$$

Power of a power: multiply the exponents. Two groups of 3 twos is 6 twos.

Zero, negative, and fraction powers

Each step to the right: exponent goes down 1, value is divided by 2



ZERO

$$2^0 = \frac{2^3}{2^3} = 1$$

NEGATIVE

$$2^{-n} = \frac{1}{2^n} \qquad 2^{-1} = \frac{1}{2}$$

FRACTION $2^{1/2} = \sqrt{2} \approx 1.414$ since $(2^{1/2})^2 = 2^1 = 2$

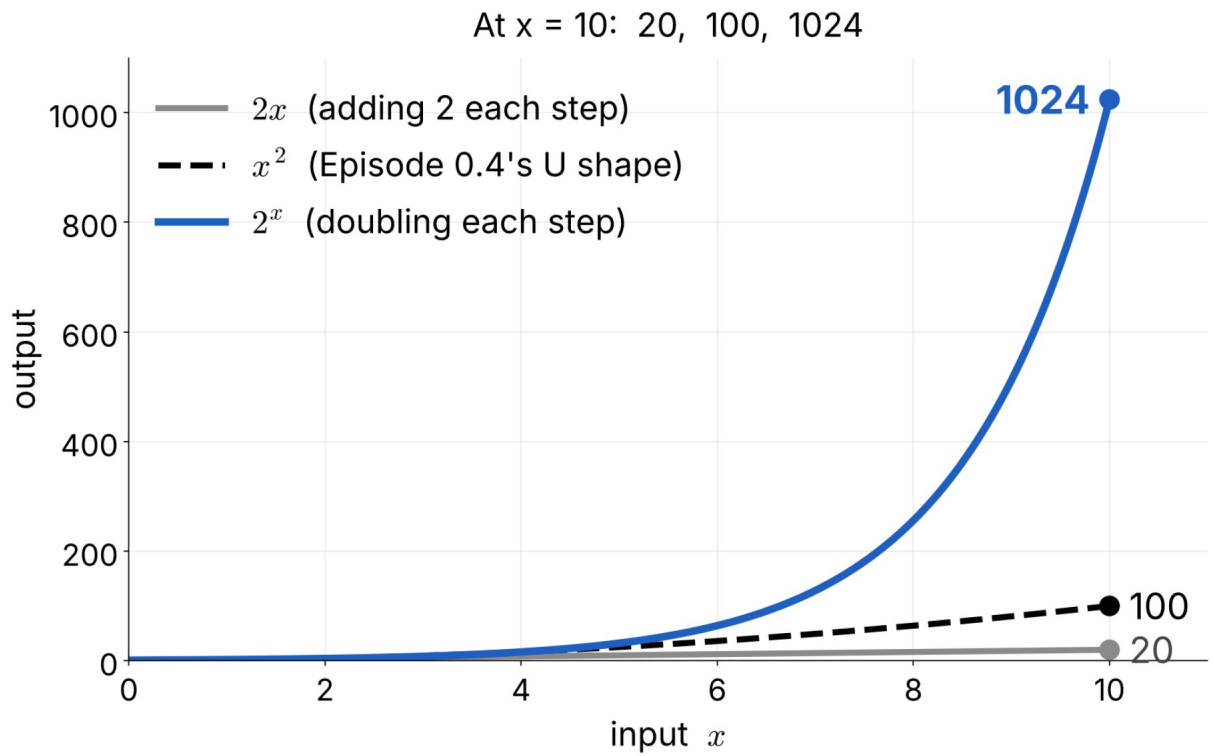
Exponents in Python

```
print(2 ** 10)       # 1024
print(2 ^ 10)        # 8  !! ^ is NOT power in Python
print(2 ** 0)        # 1
print(2 ** -1)       # 0.5
print(2 ** 0.5)      # 1.4142135623730951

thickness = 0.1
for fold in range(10):
    thickness = thickness * 2
print(thickness)       # 102.4
```

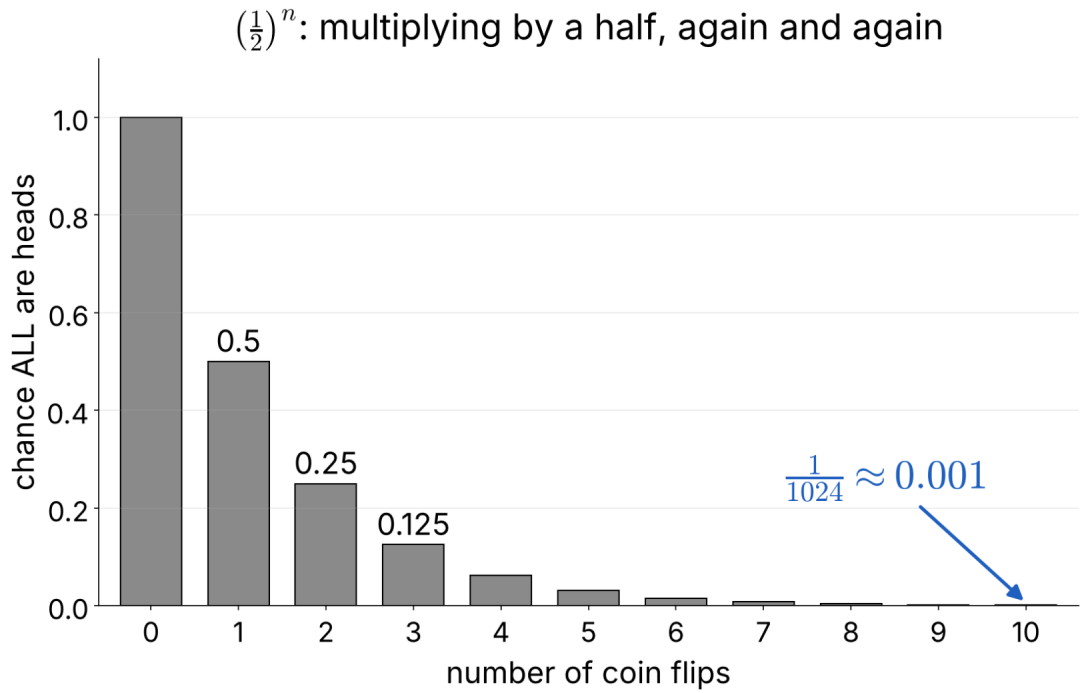
In Python, **** means “to the power of.” Never use **^**.

Doubling leaves everything behind



Adding grows steadily. Squaring grows faster. **Doubling explodes.**

Halving shrinks just as fast



10 HEADS IN A ROW

$$\left(\frac{1}{2}\right)^{10} = \frac{1}{1024} \approx 0.001$$

About **1 in 1000**.

In AI, the chance of a sentence is **many chances multiplied**. Hold that thought.

A logarithm asks the question backwards

EXPONENT: FORWARD

Multiply three 2's together. What do you get? **8**

LOGARITHM: BACKWARD

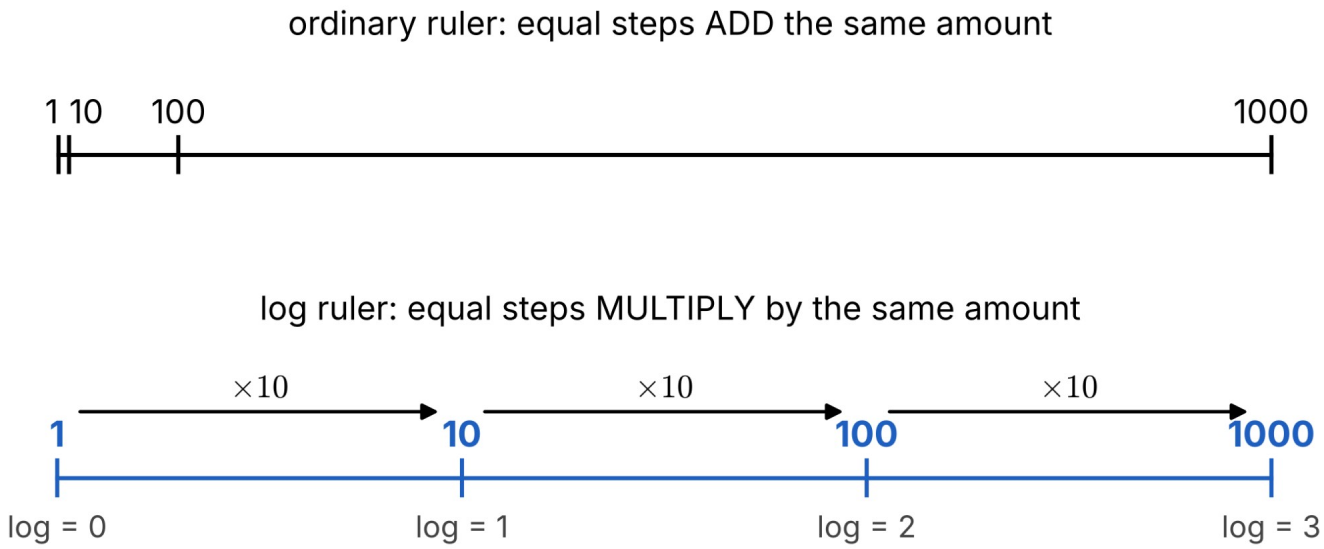
$2^{?} = 8$ How many 2's make 8? **3**

$$b^{x} = y \quad \Leftrightarrow \quad \log_b(y) = x$$

$$2^{3} = 8 \quad \Leftrightarrow \quad \log_2(8) = 3$$

Say it: "log base 2 of 8 is 3." **A log is an exponent.**

Logs count



BASE 10 COUNTS ZEROS $\log_{10}(1000) = 3$ $\log_{10}(1) = 0$ $\log_{10}(0.01) = -2$

BASE 2 COUNTS DOUBLINGS $\log_2(1024) = 10$ $\log_2(1) = 0$ $\log_2\left(\frac{1}{4}\right) = -2$

The folding bonus, solved with a log

$$0.1 \times 2^n > 1000$$ thicker than 1 meter

$$2^n > 10\,000$$ divide both sides by 0.1

$$n > \log_2(10\,000) \approx 13.29$$

“2 to the what is 10,000?” That’s a log.

ROUND UP

You can’t do 0.29 of a fold. 13 isn’t enough, so the answer is **14 folds**.

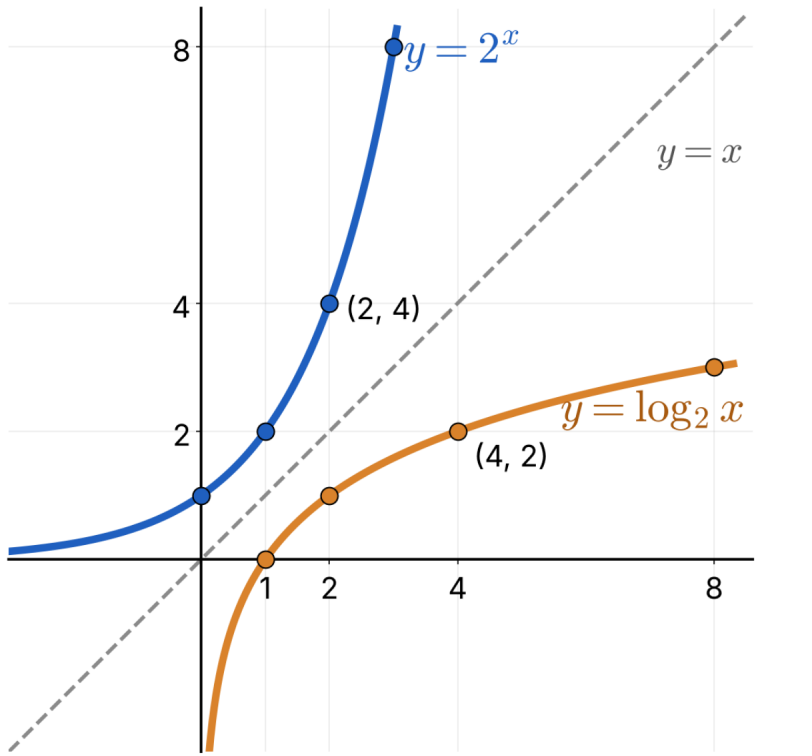
A logarithm from scratch

```
def log2(y):
    if y ⩽ 0: return None                      # no log for 0 or negatives
    low, high = -1.0, 1.0
    while high < 1024 and 2 ** high < y: high = high * 2  # widen
    while 2 ** low > y: low = low * 2
    if 2 ** low == y: return low                  # edge already right
    for _ in range(60):
        middle = (low + high) / 2             # guess the middle
        if 2 ** middle == y: return middle    # exact hit
        if 2 ** middle < y:
            low = middle                      # too small: go right
        else:
            high = middle                     # too big: go left
    return (low + high) / 2
print(log2(8), log2(10000))         # 3.0 13.28771237954945
```

Guess, check, halve the window. Repeat. Only ** needed.

The log curve is the mirror image

Swap input and output: the curve flips across y = x



2 to the x: put in the exponent, get the number.

$\log_2$ x: put in the number, get the exponent.

$\log_2 1 = 0$, because $2^0 = 1$.

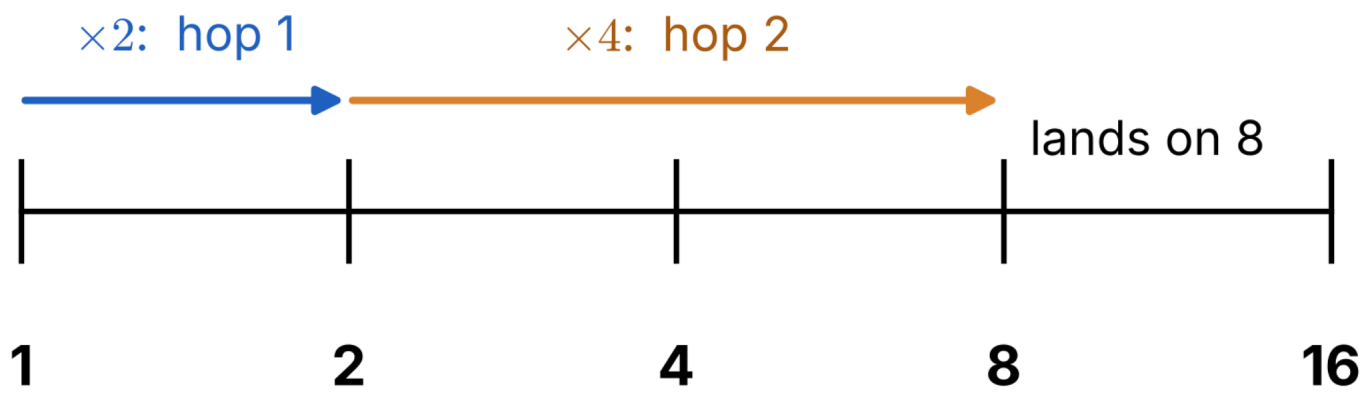
Between 0 and 1, logs are **negative**.

0 and negative numbers have **no log**.

It grows forever, but slowly: $\log_2$ of a million is only about **20**.

The superpower: multiplying becomes adding

$2 \times 4 = 8$ becomes $1 + 2 = 3$



$$\log_2 (2 \times 4) = \log_2 2 + \log_2 4 = 1 + 2 = 3$$

On a log ruler, a multiplication is just two hops added together.

Three log rules, from the three exponent rules

EXPONENT RULE		LOG RULE
$b^m \cdot b^n = b^{m+n}$	$\rightarrow$	$\log_b(xy) = \log_b x + \log_b y$
$\frac{b^m}{b^n} = b^{m-n}$	$\rightarrow$	$\log_b\left(\frac{x}{y}\right) = \log_b x - \log_b y$
$(b^m)^n = b^{mn}$	$\rightarrow$	$\log_b(x^n) = n \cdot \log_b x$

Times becomes plus. Divide becomes minus. A power becomes a times.

AI use 1: tiny chances vanish

The chance of a passage = the chances of its words, multiplied. 400 words at 0.1 each:

$$0.1^{400} = 0.000\ldots0001 \quad (400 \text{ decimal places})$$

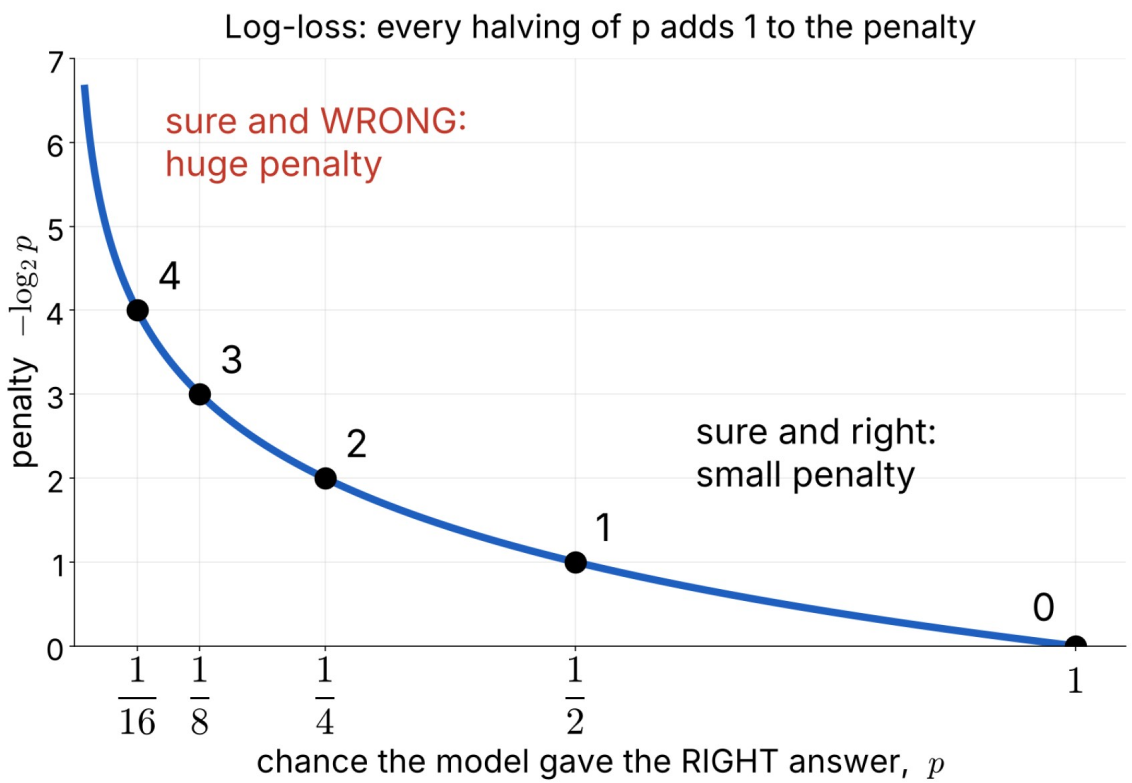
Python prints **0.0**. Not really zero: just too small to store.

THE FIX: ADD THE LOGS INSTEAD

$$\log(p_1\, p_2 \cdots p_n) = \log p_1 + \log p_2 + \cdots + \log p_n$$

$$\log_{10}\left(0.1^{400}\right) = 400 \cdot \log_{10}(0.1) = -400$$

AI use 2: log-loss



$$\text{loss} = -\log_2(p)$$

p = the chance the model gave to the **right** answer.

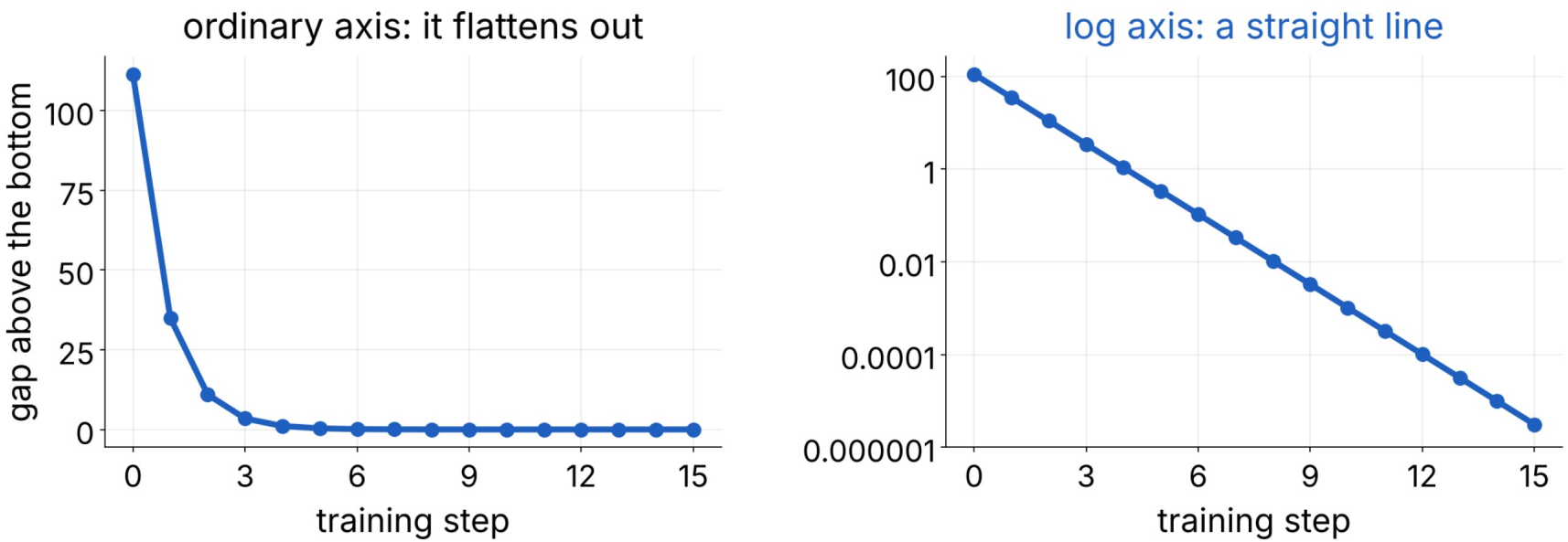
$$-\log_2\left(\frac{1}{8}\right) = -(-3) = 3$$

Every time p is **cut in half**, the penalty goes **up by 1**.

Real libraries use base e (next episode): same shape, rescaled. Also called cross-entropy.

AI use 3: our training shrank exponentially

Every step multiplies the gap by the same number: 0.3136



$$\text{gap after } k \text{ steps} = 111.36 \times 0.3136^{k}$$

Where they live in real AI

Where	What the exponent or log is doing
Log-loss	The training penalty for most classifiers and language models
Log-probabilities	Adds logs instead of multiplying tiny chances
Log-scale plots	Shows steady progress in a loss curve that looks flat
Learning rates	Tried in powers of ten: 0.1, 0.01, 0.001
Softmax	Turns scores into chances with powers of e (next episode)

Exponents make numbers explode. Logs tame them.

Your turn

1 $2^3 \cdot 2^5$ Write it as one power of 2. Then work it out.

2 $\log_2(32)$ 2 to the what is 32?

3 $p = \frac{1}{16}$ A model gave the right answer this chance. What's its log-loss, base 2?

Pause the video. Say each one out loud, then work it out.

Answers

1 $2^3 \cdot 2^5 = 2^8 = 256$ Add the exponents: 3 + 5 = 8.

2 $2^5 = 32 \quad \text{so} \quad \log_2(32) = 5$ 2, 4, 8, 16, 32: five doublings.

3 $-\log_2\left(\frac{1}{16}\right) = -(-4) = 4$ Four halvings from 1: penalty 4.

THE EQUATION OF THE DAY

$$b^x = y \quad \Leftrightarrow \quad x = \log_b(y)$$

A logarithm answers one question: what power do I need? And it turns multiplying into adding.

NEXT TIME · EPISODE 0.7

The Number e

Exponential growth, and the most useful base in AI.

YOUR CHALLENGE · answer in Episode 0.7

$$\left(1+\frac{1}{n}\right)^{n} \qquad n=1:\ 2 \qquad n=2:\ 2.25 \qquad n=12:\ ? \qquad n=365:\ ?$$

$1 at 100% interest for a year. Split the year into n parts: each part adds 100%/n of what you have now. What do you have for n = 12? n = 365? Bonus: try a million. Does it grow forever?